ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



F.CSM302 Үйлдлийн систем · Бие даалт 1

Судалгаа, хэрэгжилт, туршилтын тайлан

Kernel Data Structures

Хуваарилалтын код: **V-1-4** · Үзүүлэлт: Kernel structure graph · Бүлэг 3 – Processes

Сурах бичиг: Chapter 1 Introduction, 1.9 Kernel Data Structures (p. 36)

GitHub: github.com/grl059/OS_BD1

Хичээл заасан багш: Г. Ганбат (F.CS10)

Гүйцэтгэсэн баг: Н. Ням-Од (B222270805)
А. Гэрэл (B241940001)
А. Номин (B242270088)
Э. Мягмаршагай (B241910001)
О. Хүслэн (B241910020)
Ч. Түмэншагай (B160910830)

2026–2027 оны хичээлийн жилийн намар · Семинарын цаг: 5-1 пар
Улаанбаатар

Агуулга

Хураангуй 3
1. Удиртгал 4
2. Сурах бичгийн шаардлага 5
3. Онолын үндэс 7
4. Системийн зохиомж 8
5. Хэрэгжилт 9
6. Туршилт ба веб аппын дүрслэл 11
7. Туршилтын арга 14
8. Үр дүн 16
9. Хэлэлцүүлэг 20
10. Дүгнэлт 21
Ашигласан материал 21
Хавсралт А. Ажлын төлөвлөгөө ба гишүүдийн оролцоо 22

Хураангуй

Асуудал. Kernel процесс, нөөц, файлуудаа холбоост жагсаалт, дараалал, стек, bitmap, хэш хүснэгт, мод зэрэг өгөгдлийн бүтцээр удирддаг. Эдгээр бүтцийг сурах бичгээс унших нь тэдгээрийн шинж чанар (цаг хугацааны нарийвчлал, санах ойн зохион байгуулалт, тэнцвэржилт) ойлгоход хангалтгүй байдаг, учир нь бодит зангилаа нэмэгдэх, устгагдах тусам граф дотроо хэрхэн өөрчлөгдөж байгааг статик текстээр төсөөлөхөд хэцүү. Иймээс сурагч/оюутан бүтэц бүрийг (а) зөв хэрэгжүүлж, (б) элемент нэмэх, устгах, хайх үйлдлийн дараах төлөвийг нүдээр харуулж, (в) хэрэгжилтийн зөв ажиллагааг системтэй туршилтаар нотлох шаардлагатай тулгардаг. Энэхүү ажил яг энэ гурван шаардлагыг нэгэн зэрэг хангахыг зорьсон.

Шийдэл. Linux-ийн `list_head`-ийн next/prev семантикийг C-ээр дахин бичиж, есөн бүтэц (жагсаалт, queue, stack, bitmap, hash map, BST/AVL мод, red-black мод, min-heap, PCB бүхий ready queue ба FCFS/Round Robin scheduler) бүхий командын интерпретатор (`kds`) бүтээв. Энэ C программд `--json` горим нэмж, команд бүрийн дараах бүтцийн төлөвийг JSON хэлбэрээр гаргадаг болгов. Веб дээр ажилладаг хэрэглэгчийн интерфэйс (Next.js, `web/` хавтас) нь тэр JSON-ыг C программаас шууд авч, өөрөө юу ч тооцоолохгүйгээр зурж харуулдаг – өөрөөр хэлбэл, нэг л «бодол» (логик) байдаг: C. Веб нь түүнийг зөвхөн дэлгэцэнд үзүүлнэ. Сурах бичгийн 3-р бүлгийн Programming Project-ийн дагуу kernel module нь `struct color` жагсаалт болон `start` параметртэй Collatz дарааллын жагсаалтыг kernel орчинд үүсгэж, устгана.

Туршилт. 8 жишээ скрипт ба 5000×8 командын stress тест ASan/UBSan-тай давсан (GitHub Actions-ийн Linux job; macOS 26 дээр ASan runtime эхлэхэд гацдаг асуудлаар `make SAN=0` ашигласан). Текст гаралт ба `--json` гаралтыг нэг C программаас зэрэг гаргаж, 40 000 санамсаргүй командад (8 seed × 5000, тэр дотор AVL) мөр мөрөөрөө ижил мессеж өгснийг баталгаажуулсан. AVL-ийн өндрийн хязгаарыг 30 санамсаргүй гүйлтээр, red-black мод ба min-heap-ийн инвариантыг 40 санамсаргүй гүйлтээр шалгасан. GitHub Actions дээр C программ ба веб аппыг автоматаар build, тест хийдэг CI тохируулсан. Kernel module-ийн өмнөх хувилбар Ubuntu 24.04 (kernel 6.11.0-17) дээр insmod/rmmod хийгдсэн.

Гол үр дүн. Оролт, огноо, хязгаарыг шалгадаг, хэрэглэгч өөр өгөгдлөөр дахин туршиж болох, терминал (CLI) ба веб хуудас хоёулаа яг нэг C программаас гарсан мэдээллийг харуулдаг хэрэгсэл бүтээгдсэн. Ингэснээр сурах бичгийн 1.9-д заасан бүтэц бүрийг зөвхөн кодоор

бус, графикаар нүдээр ажиглаж, онолын цаг хугацааны нарийвчлалыг ($O(1)$, $O(\lg n)$, $O(n)$) бодит тоогоор (мод/AVL-ийн өндөр, хайлтын алхам, collision) баталгаажуулах боломжтой болсон – энэ нь абстракт алгоритм ба kernel-ийн бодит хэрэгжилтийн хоорондох зайг багасгаж буй практик ач холбогдолтой.

1. Удиртгал

Үйлдлийн системийн kernel нь процессын жагсаалт, ready queue, bitmap (нөөц чөлөөтэй эсэх), хэш хүснэгт (хурдан хайлт), модон бүтэц (scheduler, санах ой) зэргийг өгөгдлийн бүтцээр удирддаг. Эдгээр бүтэц нь зөвхөн онолын жишээ биш – kernel-ийн найдвартай, хурдан ажиллах чадвар шууд тэдний зөв хэрэгжилтээс хамаардаг тул Үйлдлийн системийн чиглэлээр суралцаж буй оюутан эдгээрийг зөвхөн уншиж ойлгоход хязгаарлагдахгүй, өөрийн гараар хэрэгжүүлж, туршиж, зөв эсэхийг нотолж байж бүрэн эээмшинэ гэж үзсэн.

Зорилго. Төслийн зорилго нь дараах дөрвийг нэгэн зэрэг хангах явдал юм:

1. Сурах бичгийн 1.9 Kernel Data Structures-д заасан бүх үндсэн бүтэц (холбоост жагсаалт, stack, queue, BST, тэнцвэржсэн мод, hash map, bitmap) болон 3-р бүлгийн Processes сэдвийн PCB/ready queue-г C хэлээр, Linux-ийн `list_head` семантиктай нийцүүлэн хэрэгжүүлэх;
2. Элемент нэмэх, устгах, хайх үйлдэл бүрийн дараах бүтцийн төлөвийг график хэлбэрээр нүдэнд харагдахуйц болгож, абстракт онолыг (цаг хугацааны нарийвчлал, тэнцвэржилт) бодит тоо, дүрсээр баталгаажуулах;
3. Хэрэгжилтийн зөв ажиллагааг санамсаргүй stress болон fuzz тестээр, мөн ASan/UBSan ашиглан системтэй нотлох;
4. Сурах бичгийн Programming Project (3-р бүлэг)-ийн дагуу бодит Linux kernel module бичиж, жинхэнэ kernel орчинд ачаалж (insmod/rmmod) ажиллагааг баталгаажуулах.

Ач холбогдол. `list_head` бол Linux kernel-ийн хамгийн их хэрэглэгддэг бүтэц бөгөөд `task_struct` (PCB), ready queue зэрэг нь түүн дээр суурилдаг [4]. Үүнийг ойлгох нь 3-р бүлгийн «Processes» сэдвийн суурь болно [1]. Практик талаас нь авч үзвэл, kernel дотор ашигладаг *intrusive* (өгөгдлийн struct дотроо холбоос агуулсан) жагсаалтын загвар нь хэрэглээний программ дээр (user-space) ердийн хэллэгт заадаг жагсаалтаас (жишээ нь зөвхөн утга хадгалдаг жагсаалт) бүтцийн хувьд ялгаатай тул оюутанд энэ ялгааг гараар хэрэгжүүлж үзэх нь ач холбогдолтой. Боловсролын

талаас нь, график дүрслэл нь *тэнцвэржсэн* (balanced) ба *тэнцвэргүй* (unbalanced) модны хайлтын гүнзгийрэлтийн ялгааг, мөн хэш хүснэгтийн *collision*-ийн нөлөөг шууд нүдээр харуулдаг тул онолын хичээлээс гадна практик ойлголт бэхжүүлэх зорилготой.

Хамрах хүрээ. Хэрэглэгчийн орчны загвар (C, CLI), түүн дээр суурилсан веб хуудас (Next.js), мөн нэг kernel module. Бодит Linux scheduler-т холбогдохгүй – өөрөөр хэлбэл, scheduler-ийн алгоритм (FCFS, Round Robin) нь Linux-ийн бодит CFS scheduler-ийг орлохгүй, зөвхөн сурах бичгийн энгийн загварыг тайлбарлахад зориулагдсан.

2. Сурах бичгийн шаардлага

Хуваарилалт: код 4, «Kernel Data Structures», үзүүлэлт «Kernel structure graph», бүлэг 3 – Processes, F.CSM302 «Бие даалт 1» удирдамжийн дагуу [5]. Онолын хэсэг нь **Chapter 1 Introduction-ийн 1.9 Kernel Data Structures (p. 36)**. **Хүрэх үр дүн:** «Сурах бичигт заасан kernel data structure-ийн хэрэглээг хэрэгжүүлж, элемент нэмэх, устгах, хайх үйлдлийн дараах бүтцийг графаар харуулна.»

Operating System Concepts-ийн 3-р бүлгийн Programming Project «Kernel Data Structures» нь Chapter 1-ийн 1.9 Kernel Data Structures (p. 36)-д тайлбарласан бүтцүүд дээр тулгуурласан хоёр хэсэгтэй:

- **Part I:** `struct color {red, blue, green; struct list_head list;}` төрлийн элементүүдийг kernel module дотор жагсаалтад нэмж, `list_for_each_entry`-ээр гүйлгэн хэвлэх, module-ийг буулгахад `list_for_each_entry_safe`-ээр устгаж санах ойг чөлөөлөх.
- **Part II:** module-д `start` параметр дамжуулж (`module_param`), Collatz дарааллыг (тэгш бол n/2, сондгой бол 3n+1, 1 хүртэл) холбоост жагсаалтад хадгалан хэвлэх, буулгахад устгах.

Удирдамжийн шаардлага бүр хэрэгжсэн эсэхийг баталгаажуулах, мөн шалгагч хүн тус бүрийг хурдан хайж олох боломжтой байлгахын тулд доорх хүснэгтэд удирдамжийн заалт, хэрэгжүүлсэн файл/функц, туршсан үйлдэл, баталгаажуулсан тестийг нэг мөрөнд эгнүүлэн харуулав.

Шаардлага	Хэрэгжилт	Нэмэх / устгах / хайх	Баталгаа
Ch.3 Part I: struct color жагсаалт	`kds_module.c`, `color_list`	`list_add_tail`, `list_del`, `list_for_each_entry`	`run_module_test.sh`, dmesg
Ch.3 Part II: Collatz, `start` параметр	`kds_module.c`, `module_param(start)`, 1..1 000 000	`list_add_tail`, `list_del`	start=25, 15, 0 (татгалзана)
1.9.1 Холбоост жагсаалт (list_head)	Давхар холбоост дугуй жагсаалт (struct birthday)	add_head, add_tail, del, find	textbook, search, веб «Чөлөөт горим»
1.9.1 Stack (LIFO)	list_head, дээд талаас нэмж авна	push, pop, find	all_structures, search
1.9.1 Queue (FIFO)	list_head, төгсгөлд нэмж эхнээс авна	push, pop, find	all_structures, search
1.9.2 Binary search tree	Энгийн BST (хамгийн муу $O(n)$)	insert, delete, find	search: 1–5 → өндөр 5
1.9.2 Тэнцвэржсэн мод ($O(\lg n)$)	AVL (`tree mode avl`)	insert, delete, find	search, avl_check: 1–7 → өндөр 3
1.9.3 Хэш функц ба hash map	Кодын нийлбэр mod n; collision-д bucket доторх hlist_head жагсаалт; ачаалал > 0.75 бол bucket ×2 (`hash auto on`), гараар `rehash`	put, del, get, rehash, stats	search, hash_resize
Нэмэлт: red-black мод (CFS-ийн мод)	`rb` горим: өнгө, black-height, эргүүлэлтийн тоо	insert, delete, find	rb_heap, rb_check (30 seed)
Нэмэлт: min-heap (priority queue)	массив дээрх овоолго, багтаамж 16, swap тоо	push, pop, peek	rb_heap, rb_check (10 seed)
1.9.4 Bitmap	n = 1..128 бит	set, clear, test	all_structures, errors
«Processes» холбоос	PCB + list_head ready queue, FCFS, Round Robin	proc new / дуусах / sched print	scheduler, scheduler_full

Хүснэгтээс харахад удирдамжийн 1.9-д заасан бүх үндсэн бүтэц (жагсаалт, stack, queue, BST, тэнцвэржсэн мод, hash map, bitmap) болон «Processes» сэдвийн PCB/ready queue холбоос бүрэн хэрэгжсэн бөгөөд Ch.3-ийн Part I/II-ийн kernel module даалгавар хоёулаа тусдаа баталгаажуулалттай (dmesg, скрипт) байна. Нэмэлтээр хичээлийн агуулгыг гүнзгийрүүлэх зорилгоор red-black мод, min-heap-ийг мөн хэрэгжүүлж, тус тусдаа тестээр нотолсон.

Харуулах шаардлага: веб хуудас нь бүтэц бүрийн нэмэх, устгах, хайх үйлдлийн дараах төлөвийг граф хэлбэрээр шууд зурдаг (6-р хэсэг).

3. Онолын үндэс

Эх сурвалж: Silberschatz, Galvin, Gagne, *Operating System Concepts*, 10th ed., Chapter 1 Introduction, 1.9 Kernel Data Structures, p. 36 (1.9.1 жагсаалт/stack/queue, 1.9.2 мод, 1.9.3 hash, 1.9.4 bitmap) [1].

list_head нь *intrusive* (өгөгдлийн struct дотроо холбогдох талбараа агуулсан), давхар холбоост, дугуй жагсаалт юм. Node нь өгөгдлийн struct дотор `list_head` талбар агуулж, `container_of` макро нь талбарын хаягаас эзэн struct-ийг сэргээнэ. Нэмэх, устгах үйлдэл $O(1)$, хайх нь $O(n)$ [4]. Доорх Зураг 1a нь гурван зангилаатай ийм жагсаалтын `next`/`prev` холбоосыг харуулав.

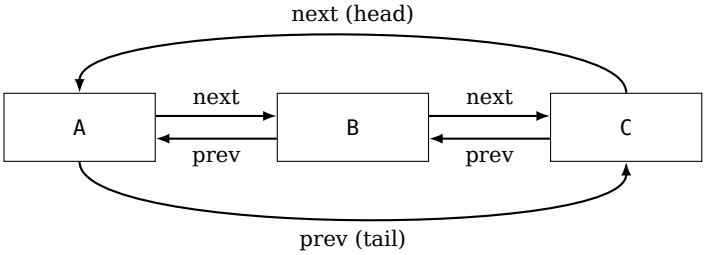
Queue (*FIFO*, First-In-First-Out) нь tail-д нэмж head-ээс авдаг. Kernel-ийн ready queue ийм зарчмаар ажилладаг. **Stack (*LIFO*, Last-In-First-Out)** нь head-д нэмж head-ээс авдаг [2].

Bitmap-д 1 бит нь 1 нөөцийг илэрхийлнэ. set/clear/test нь $O(1)$ бөгөөд Boolean массиваас 8 дахин бага зай эзэлнэ [1].

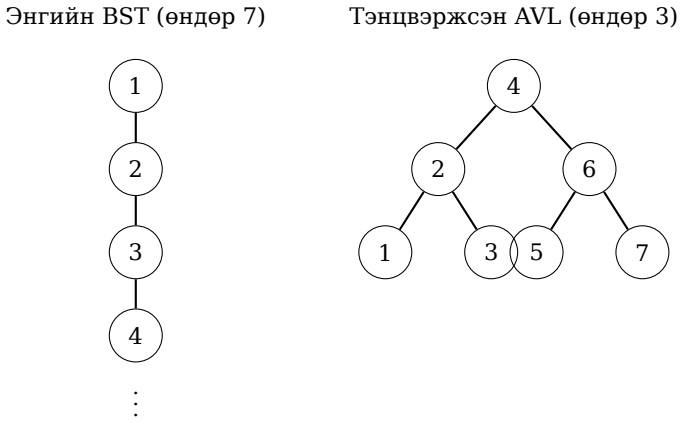
Binary search tree (*BST*)-д зүүн $<$ үндэс $<$ баруун байна. Дундаж хайлт $O(\lg n)$, харин эрэмбэлсэн оролтод мод нэг шугам болж хамгийн муу тохиолдол $O(n)$ болно. *Тэнцвэржсэн* (*AVL*) мод n элементэд ойролцоогоор $\lg n$ түвшинтэй байдаг [2]. Linux red-black мод ашигладаг бол бид ижил зорилготой AVL модоор (өндрийн зөрүү ≤ 1, эргүүлэлт) загварчилсан – AVL тэнцвэржилтийн анхны алгоритмыг Adelson-Velsky ба Landis тодорхойлсон [3]. Доорх Зураг 1b нь ижил 1–7 оролтод энгийн BST ба AVL-ийн өндрийн ялгааг харуулав.

Hash map-д хэш функц түлхүүрийг хүснэгтийн индекс болгож, ихэнх тохиолдолд $O(1)$ хайлт өгнө. Хоёр түлхүүр нэг индекст таарвал (*collision*) тухайн bucket-д жагсаалт залгана. Collision олшрох тусам хайлт $O(n)$ руу ойртоно [2].

Процесс ба scheduler. Процесс бүр *PCB* (Process Control Block)-тэй байдаг (Linux-д `task_struct`). Бэлэн процессууд *ready queue*-д зогсоно. *FCFS* (First-Come-First-Served) нь процессыг дуустал нь ажиллуулна. *Round Robin* нь процесс бүрд квант хугацаа өгч, дуусаагүй бол дарааллын төгсгөлд буцаана. Хүлээлтийн хугацааг нийт үргэлжилсэн хугацаанаас ажлын хугацааг хасаж олно [1].



Зураг 1a – list_head: гурван зангилаа next/prev холбоосоор дугуй болж холбогдсон байдал.



Зураг 1b – ижил 1–7 эрэмбэлсэн оролтод энгийн BST нэг шугам (өндөр 7) болдог бол AVL тэнцвэржилтээрээ өндрөө 3-т хадгална.

Kernel module ба параметр. `module_param(name, type, perm)` нь `insmod kds_module.ko start=15` хэлбэрээр утга дамжуулах боломж олгоно. Init амжилтгүй болбол `module_exit` дуудагдахгүй тул init өөрөө нөөцөө чөлөөлөх ёстой. Нийтлэг алдаанууд: `list_del`-ийн дараа pointer ашиглах, node чөлөөлөхгүй үлдээх (leak), гүйлгэж байхдаа устгах (энэ үед `_safe` хувилбар хэрэгтэй).

4. Системийн зохиомж

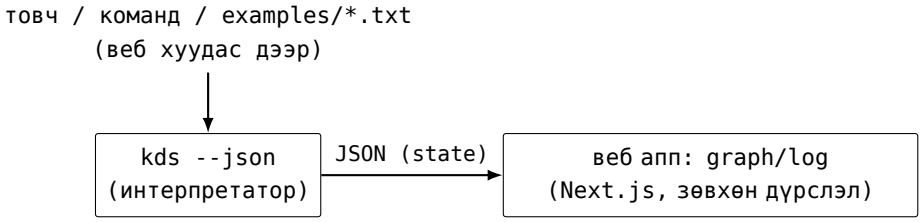
Компонентууд:

- **(a) src/user/kds.c** – командын интерпретатор ба 9 бүтэц (жагсаалт, queue, stack, bitmap, hash, BST/AVL мод, red-black мод, min-heap, процесс/хуваарилагч). Жагсаалт, queue, stack, hash-ийн bucket, ready queue бүгд нэг `list_head` хэрэгжилтийг ашиглана. Энэ файл дотор ердийн текст мессеж гаргадаг горимоос гадна `--json` горим бий: команд тус бүрийн дараа бүтцийн бүх төлөвийг нэг JSON мөр болгож бичдэг.
- **(b) web/** – Next.js (React, TypeScript) дээр бичигдсэн веб апп. Хэрэглэгч товч дарах эсвэл команд бичих үед веб апп сервер талд `kds --json`-ыг

ажиллуулж, буцаж ирсэн JSON мөр бүрийг дэлгэц дээр граф болгож зурна. Веб апп өөрөө ямар ч бүтцийн логик (нэмэх, устгах, хайх дүрэм) агуулдаггүй – зөвхөн C программаас ирсэн төлөвийг дүрсэлж харуулна. Нүүр (7 шатны жагсаалт ба дээд цэс), «Чөлөөт горим» (бүтэц сонгож, үйлдлийн товчоор турших), «Хичээл» (7 шат: алхам бүрт Багшийн тайлбар ба монгол дуу, асуулттай тест), «Тухай» хуудастай.

- **(c) `src/kernel/kds_module.c`** – kernel орчинд ажиллах хувилбар: color, Collatz, birthday жагсаалт, kfifo, bitmap, мөн Linux-ийн `<linux/hashtable.h>`, `<linux/rbtree.h>` систем толгой файлуудыг ашигласан hash хүснэгт, red-black мод ба `/proc/kds` интерфэйс.
- **(d) `tests/`** – C-ийн текст гаралт ба `--json` гаралтыг хооронд нь, мөн хүлээгдэх гаралттай харьцуулдаг тестүүд.

Өгөгдлийн урсгал: хэрэглэгчийн команд (товч, командын мөр эсвэл script) → веб апп серверийн тал нь командыг `kds --json`-д дамжуулна → C программ бүтцийн төлөвийг шинэчилж, нэг JSON мөр болгож буцаана → веб апп тэр JSON-оос шууд график зурна. Тооцоолол бүгд C дотор хийгддэг тул веб дээр харагдах зураг бодит ажиллагаанаас хэзээ ч зөрөхгүй.



Өгөгдлийн урсгал: команд C программ руу, төлөв JSON хэлбэрээр веб рүү.

5. Хэрэгжилт

Хэл ба хэрэгсэл: C (gcc, `-Wall -Wextra -fsanitize=address,undefined`), JavaScript (HTML/SVG, сан ашиглаагүй), kernel C (Kbuild), туршилтад Python 3 ба Node.js.

Гол шийдлүүд:

1. `list.h`-ийн `__list_add`, `list_add`, `list_add_tail`, `list_del`, `list_for_each(_safe)`, `container_of`-ийг өөрсдөө бичсэн.
2. Queue ба stack нь ижил list_head-ийг хуваалцах бөгөөд зөвхөн нэмэх талаараа ялгарна. Багтаамж нь 8.
3. Огноо шалгахдаа өндөр жилийг тооцно: 4-т хуваагддаг ба 100-д хуваагддаггүй, эсвэл 400-д хуваагддаг.

4. Модноос хоёр хүүтэй node устгахдаа баруун дэд модны хамгийн бага утгаар орлуулна. AVL горимд буцах замдаа `bal()` нь rotl/rotr эргүүлэлтээр тэнцвэржүүлнэ. Модны төрлийг зөвхөн хоосон үед солино.

5. Hash map нь 1–16 bucket-тэй. Хэш функц нь тэмдэгтүүдийн кодын нийлбэр mod n. Collision-ийг bucket доторх hlist_head (singly холбоост) жагсаалтаар шийднэ (chaining). Нийт 24 элементээр хязгаарласан. `get` үйлдэл хайлтын алхмын тоог мэдээлнэ.

6. PCB нь `struct pcb {pid, name, rem, slice, list_head}` бүтэцтэй. Ready queue-д `list_add_tail`-аар нэмж, эхний элементийг `list_del`-ээр авч CPU-д өгнө. `sched tick` бүрд CPU 1 тик ажиллана. FCFS нь процессыг дуустал ажиллуулна. RR-д квант дуусвал процесс дарааллын төгсгөлд буцна (preempt). Амьд процессын тоо 8 хүртэл.

7. Хайх үйлдэл бүх бүтцэд байна: list find (индекс), queue/stack find (байрлал), tree find, bitmap test, hash get.

8. **Kernel module:** init дотор color (4 элемент), Collatz (start-аас 1 хүртэл, хамгаалалтын хязгаар 1000 гишүүн), birthday, kfifo, bitmap-ийг үүсгэнэ. `start` нь 1..1 000 000 мужаас гадуур бол `-EINVAL` буцааж module ачаалагдахгүй. `kmalloc` алдаа гарвал `goto fail`-аар бүх жагсаалтыг цэвэрлэнэ. Exit дотор `list_for_each_entry_safe`-ээр элемент бүрийг хэвлэж устгана.

9. Веб апп нь C программыг дахин бичсэн хуулбар биш. Нэг л дүрэм, нэг л код (`kds.c`) байдаг бөгөөд веб зөвхөн түүний `--json` гаралтыг уншиж зурдаг. Иймээс «веб ба C-ийн гаралт зөрөх» эрсдэл угаасаа байхгүй – тэд угаасаа нэг эх сурвалжтай.

5.1 GitHub дээрх ажиллагаа

Repository: `github.com/grl059/OS_BD1` (салбар `main`). Бүтэц нь удирдамжийн 6-р хэсгийн дагуу: `README.md` (нэр, код, зорилго, Quick Start, AI тэмдэглэл), `src/`, `tests/`, `examples/`, `docs/` (Manual, Report, screenshot, тайлангийн эх LaTeX ба HTML). Build-ийн гаралт болон гарын үсгийн түлхүүрийг `.gitignore`-оор хассан. 9-р долоо хоногт нийтэлсэн хувилбар нь `v1.0.0` tag ба GitHub Release. Тайлан, Manual-ийн **албан ёсны эх** нь `docs/latex/report.tex`, `manual.tex` (XeLaTeX, DejaVu фонт) бөгөөд `sh docs/latex/build\_latex.sh` нь эцсийн `docs/V-1-4\_BD1\_Report.pdf`, `docs/V-1-4\_Manual.pdf`-ийг үүсгэнэ; `docs/report\_src/` нь HTML нөөц эх, `build\_pdf.py` нь мөн ижил нэрээр гаргана.

Хүснэгтийн зорилго: доорх commit-ийн түүх нь төслийг нэг удаагийн хуулбар бус, долоо хоногоор ахиж хөгжсөнийг нотлох баримт болгон

оруулсан – огноо, commit-ийн нэр, агуулга гурвыг зэрэг харуулж, аль үе шатанд аль хэсэг (UI, core код, kernel module, тестийн скрипт) нэмэгдсэнийг шалгуулагч шууд хянах боломжтой.

Огноо	**Commit**	**Агуулга**
2026-10-20	first commit (×2), Change1	Repository үүсгэв; эх код, жишээ, тест, анхны UI, README
2026-10-20	UI arai oiglomjtoi bolgov	UI-г энгийн хэрэглэгчид ойлгомжтой болгосон (галт тэрэг, дараалал г.м. зүйрлэл)
2026-10-25	Change2, ui	Hash, AVL, scheduler tab-уудыг UI-д нэмсэн
2026-10-25	Report zasav, Delete kds, Example	Тайлан, Manual шинэчилсэн; binary файлыг хассан
2026-10-25	Code buten bolgov	kds.c-д hash/AVL/scheduler/find; шинэ жишээ ба тест; kernel module Part I/II; README, Manual, Report засвар
2026-10-26…27	JSON mode, web app, CI	`kds.c`-д `--json` горим нэмсэн; хуучин нэг файлтай UI (`docs/ui/index.html`)-ийг устгаж, оронд нь Next.js веб апп (`web/`) бичсэн; GitHub Actions-аар автомат build/тест (CI) ба release тохируулсан

Хүснэгтээс харахад ажил 2026-10-20-ноос 2026-10-27 хүртэл тасралтгүй үргэлжилсэн бөгөөд эхний commit-үүд үндсэн бүтэц, UI-г бэлтгэж, дундын шат (10-25) hash/AVL/scheduler болон kernel module-ийг нэмж, эцсийн commit-үүд (10-26…27) `--json` горим, Next.js веб апп ба CI/release-ийг бататгасан. Энэ дараалал нь шаардлагын хэрэгжилт баримтжуулалтын (README, Manual, Report) өөрчлөлттэй зэрэгцэж явсныг харуулна. Гишүүн бүрийн хариуцсан ажил, түүний үр дүнгийн файлыг Хавсралт А-д үзүүлэв; commit/push-ийг нэг гишүүн төвлөрүүлж хийснийг тэр хавсралтанд тэмдэглэв.

6. Туршилт ба веб аппын дүрслэл

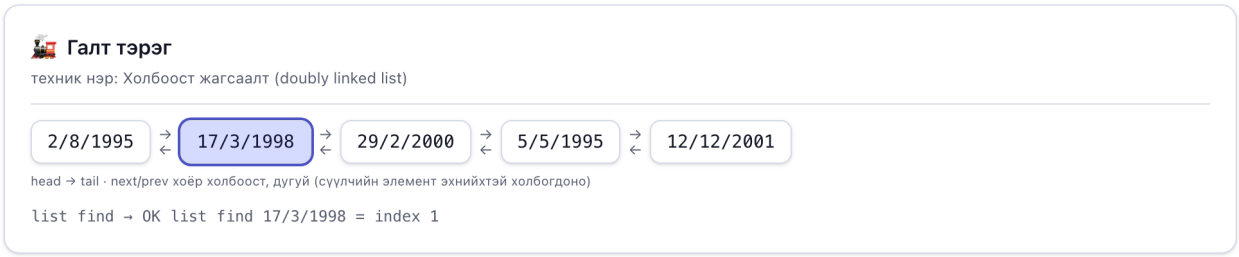
Веб апп (`web/`, Next.js) дараах хуудастай: **Нүүр** (7 шатны жагсаалт ба дээд цэс), **Чөлөөт горим** (бүтэц сонгох жагсаалт, «Үр дүн» карт, «Үйлдэл сонгох» товч, «Юу болов?» ба «Түүх (n)» самбар, «Сүүлчийнхийг буцаах», «Команд хуулах», «Шинээр эхлэх» үйлдэл), **Хичээл** (7 шат: Багш алхам бүрт тайлбар хэлж, нэг командыг тус тусад нь ажиллуулна; тайлбарыг **монгол дуугаар** сонсох дууны товчтой (чанга яригч дүрс); шат дуусвал ✓-ээр тэмдэглэгдэж хадгагдана; сүүлд нь ойлголт шалгах асуулт), **Тухай**. Өнгөний загвар нь индиго / нил ягаан. `rb` (red-black) ба `heap` (min-heap) бүтцүүд C цөм ба `--json` төлөвт бэлэн боловч вэб дэлгэцээрээ харуулаагүй; README-ийн Known issues-д заасан.

Юуг өөрчилж турших вэ: огноо, тоо, bitmap-ийн хэмжээ (1–128),

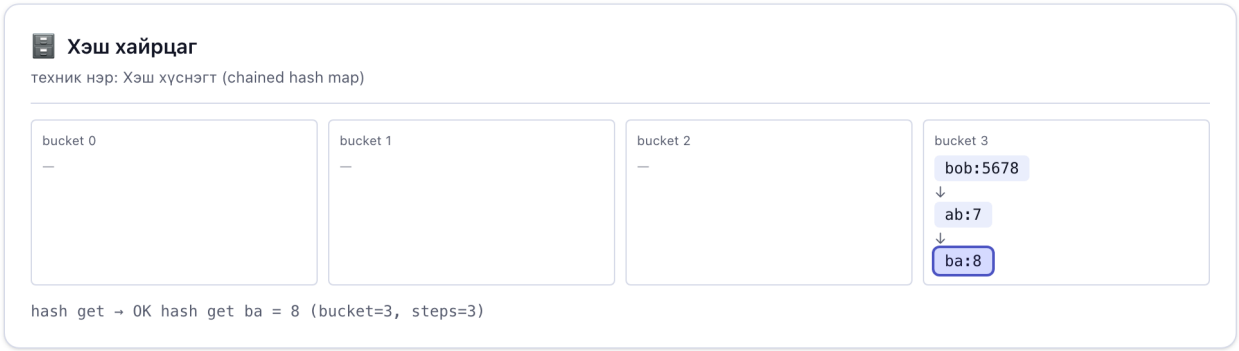
bucket-ийн тоо (1–16), түлхүүр/утга, модны төрөл (BST/AVL), процессын нэр, ажлын хугацаа (1–20), scheduler-ийн бодлого ба квант (1–10). Өгөгдлийг командын мөр эсвэл script файлаар (`examples/*.txt`) оруулна. «Шинээр эхлэх» товч ажиллуулсан командуудыг устгаж бүтцийг reset хийнэ.

Юуг ажиглах вэ: сүүлийн үйлдэлд өөрчлөгдсөн эсвэл олдсон элементийг граф дээр улбар шараар тэмдэглэнэ. Ажлын түүхэнд OK/ERR мессеж бүр жагсаалтаар харагдана. Хэш хүснэгтэд collision-ийн тоо болон хайлтын алхам, модонд өндөр, scheduler-т процесс бүрийн явцыг ажиглана. Нэг команд оруулах болгонд веб апп нь дэвсгэр дээр `kds --json`-ыг дуудаж, буцаж ирсэн JSON-ыг шууд зурдаг тул дэлгэц дээр харагдах зураг C программын бодит тооцооноос хэзээ ч зөрөхгүй.

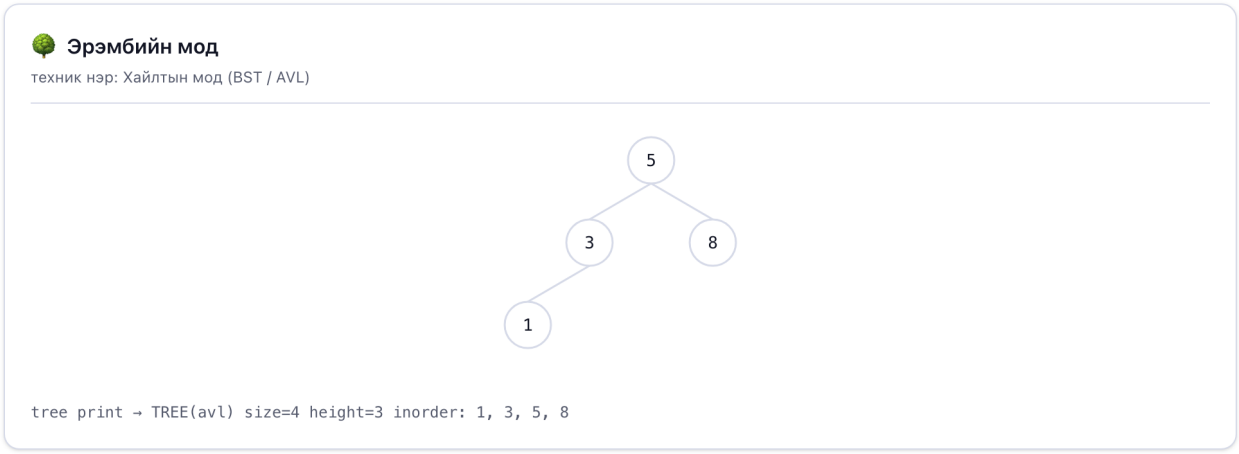
Доорх зургууд нь веб аппын бодит дэлгэцээс (Chrome) авсан бөгөөд зураг бүр `kds --json`-ийн буцаасан сүүлийн `state`-ийг шууд харуулна. Сүүлийн үйлдэлд өөрчлөгдсөн эсвэл олдсон элементийг улбар шар хүрээгээр тодотгосон. Зургийг тайлангийн энгийн текстийн фонттой (12pt) аль болох ойртуулахын тулд дэлгэцийн зургийг хуудасны бүтэн өргөнөөр (`width=1.0\textwidth`) хэвлэв.



Зураг 2 – Холбоост жагсаалт: 5 төрсөн өдрийг `add_tail`-аар дараалуулан оруулж, дараа нь `list find 17 3 1998`. Олдсон элемент (index 1) тодотгогдож, зангилаа бүрийн хооронд next/prev хоёр холбоос харагдана.



Зураг 3 – Хэш хүснэгт (4 bucket): `bob`, `ab`, `ba` нэг bucket-д унаж collision үүсэв; bucket 3 дотор chaining жагсаалт ба `hash get ba`-ийн steps=3 харагдана.



Зураг 4 – AVL мод: 5, 3, 8, 1 оруулсны дараах граф; ирмэг (parent–child) ба inorder 1, 3, 5, 8. Энгийн BST-д ижил дараалал нэг шугам болдог бол AVL өндрийг 3-д барина.



Зураг 5 – Round Robin (q=2): running P3, ready queue P1 → P2, процесс бүрийн progress bar (rem/burst), time=5, done=0. Ready queue нь `list_head` тул сум нь дараагийн PCB-г заана.

7. Туршилтын арга

Орчин: Ubuntu 24.04, gcc 13, GNU make, Node.js 22 (веб апп, CI), Python 3.13 (тест скрипт). Kernel module-д Ubuntu 24.04, kernel 6.11.0-17-generic, linux-headers.

Хүснэгтийн зорилго: доорх хүснэгт тест бүрийн оролт/нөхцөлийг хүлээгдэх үр дүнтэй хослуулж, аль тест аль бүтэц/шаардлагыг хамгаалж байгааг (жишээ нь `errors` – буруу оролт, `fuzz` – санамсаргүй довтолгоо, `avl_check` – тэнцвэржилтийн инвариант) ил харуулахын тулд оруулсан. Нэг мөр бүр тусдаа, давтагдахгүй шалгуур учраас тестийн хамрах хүрээг энгийн тоймлолоос илүү нарийн нотолно.

Өгөгдөл ба хүлээгдэх үр дүн (хүлээгдэх гаралт `tests/expected_*.out`-д хадгалагдсан):

Тест	Оролт / нөхцөл	Хүлээгдэх үр дүн
textbook	examples/textbook.txt – сурах бичгийн 5 төрсөн өдөр, гүйлгэх, бүгдийг устгах	Оруулсан дараалаар жагсаж, эцэст нь хоосон
all_structures	queue, stack, bitmap, tree	FIFO/LIFO дараалал, bitmap 1000000000000001, inorder 20, 40, 50, 70
errors	31/4, 29/2/1900, сар 13, жил 1800, налуу зураастай огноо (0/5/2000), өдөр/сар 0, хоосноос pop, size 0/129, индекс -1/99, давхардсан утга	Мөр бүр ойлгомжтой ERR, програм унахгүй
search	Бүх бүтцэд хайлт, hash collision, BST 1–5, AVL 1–7	index/position/steps, BST өндөр 5, AVL өндөр 3
scheduler	RR q=2 ба FCFS: A=5, B=3, C=2; буруу нэр, ажил, бодлого	Гараар тооцсон хуваарьтай таарна (8-р хэсэг)
scheduler_full	9 процесс үүсгэх	9 дэх нь татгалзагдана, нэг нь дууссаны дараа зөвшөөрөгдөнө
stress	5000 × (queue/stack push+pop, hash put+del, proc new+tick)	ERR 0, ASan/UBSan мэдээлэлгүй
text == –json	tests/json_check.py: 8 скриптийг текст горим ба `--json` горимоор зэрэг ажиллуулах	Мессеж бүр мөр мөрөөрөө ижил, state бүр 9 бүтцийг агуулна
fuzz	tests/fuzz_json.py: 8 seed × 5000 санамсаргүй команд (буруу огноо, том индекс, 8-аас олон процесс, урт түлхүүр г.м.)	Текст ба `--json` горимын мессеж ижил
avl_check	tests/avl_check.py: 30 санамсаргүй гүйлт (50–400 insert/delete)	Inorder = эрэмбэлсэн олонлог, өндөр $\leq 1.44 \cdot \log_2(n+2)$
hash_resize	examples/hash_resize.txt – ачаалал 0.75-аас хэтрэхэд bucket 4→8→16 auto resize, rehash/stats	Resize ба stats зөв; rehash 65/0, auto maybe мөрүүд ERR
rb_heap	examples/rb_heap.txt – red-black-д AVL-тэй ижил түлхүүр, heap push/pop/peek	Эргүүлэлтийн дараах өнгө/black-height, inorder зөв; heap-ийн дараалал, давхардал/underflow ERR
rb_check	tests/rb_check.py: red-black 30 seed + min-heap 10 seed	Өнгө, black-height, inorder эрэмбэ, heap-ийн эс хазайх чанар бүх гүйлтэд хүчинтэй
kernel module	src/kernel/run_module_test.sh: start=25, start=15, start=0	24 ба 18 гишүүнтэй Collatz, start=0 үед $-EINVAL$

Хүснэгтээс харахад 14 тест нь нэг бол оролтын (errors, search) эсвэл алгоритмын (scheduler, avl_check) зөв ажиллагааг, эсвэл веб ба C-ийн гаралтын нийцлийг (text == –json, fuzz) тус тус баталгаажуулж, хоорондоо давхцдаггүй тул 7-р хэсгийн тестийн хамрах хүрээ бүрэн, холбогдох бүх шаардлагад тулгуурласан болохыг харуулж байна. Fuzz тест алдааг илрүүлж чадаж байгаа эсэхийг шалгахын тулд C-ийн AVL-ийн давхар

эргүүлэлтийг зориуд устгаж туршсан. Ингэхэд seed 2 ба 4 нь модны өндрийн зөрүүг шууд илрүүлсэн тул тест үр дүнтэй болох нь батлагдсан.

8. Үр дүн

Хүснэгтийн зорилго: 7-р хэсгийн тест бүрийг бодитоор ажиллуулсан эцсийн гаралтыг энд нэгтгэж, PASS/FAIL төлвийг шалгуулагчид нэг дороос харуулахын тулд оруулсан.

Тест	Үр дүн
textbook, all_structures, errors, hash_resize, rb_heap	**PASS ×5**
search (хайлт, hash, BST/AVL)	**PASS**
scheduler (RR, FCFS, алдаа), scheduler_full (8 процесс)	**PASS ×2**
stress (5000 × 8 команд)	**PASS, ERR 0, sanitizer мэдээлэлгүй**
text == –json (8 скрипт)	**PASS ×8**
fuzz text == json (8 seed × 5000 команд, тэгш seed-д AVL)	**PASS ×8 (seed бүрд ≈4 700 гаралтын мөр)**
avl_check (30 гүйлт + эрэмбэлсэн 1..15)	**PASS ×32 (BST өндөр 15, AVL өндөр 4)**
rb_check (red-black 30 seed + min-heap 10 seed: өнгө, black-height, inorder/эс хазайх чанар)	**PASS ×40**
Kernel module, баталгаажуулсан хувилбар (birthday, kfifo, bitmap)	**PASS: Ubuntu 24.04, kernel 6.11.0-17, insmod/rmmod (Зураг 1)**
CI (GitHub Actions)	**PASS: C цөм (Ubuntu+macOS) ба Next.js веб апп build+тест**
Kernel module-ийн шинэчлэгдсэн хэсэг (Linux-ийн hashtable.h, rbtree.h толгой файлууд, /proc/kds)	Жинхэнэ Linux дээр `run_module_test.sh`-ээр баталгаажуулна (CI Linux job build л хийнэ). Хүлээгдэх гаралтыг доор үзүүлэв.

Хүснэгтээс харахад 9 бүтэц, scheduler, kernel module, CI бүгд PASS буцаасан бөгөөд sanitizer (ASan/UBSan) ямар ч алдаа мэдээлээгүй нь санах ойн алдаа, бүхэл тоон халихаас хамгаалагдсаныг баталгаажуулж байна. Зөвхөн сүүлийн мөрөнд заасан kernel module-ийн шинэчлэгдсэн хэсэг (Linux-ийн hashtable.h, rbtree.h толгой файлууд, /proc/kds) нь жинхэнэ Linux орчинд `insmod`-оор баталгаажаагүй (CI build л хийдэг тул), энэ нь §9-д дурдсан хязгаарлалт.

Сурах бичгийн жишээний бодит гаралт (examples/textbook.txt)

```
OK list add_tail 2/8/1995 (size=1)
OK list add_tail 17/3/1998 (size=2)
OK list add_tail 29/2/2000 (size=3)
```

```
OK list add_tail 5/11/2001 (size=4)
OK list add_tail 30/12/2003 (size=5)
LIST size=5: 2/8/1995 -> 17/3/1998 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 17/3/1998 (size=4)
LIST size=4: 2/8/1995 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 2/8/1995 (size=3)   ...   OK list del 30/12/2003 (size=0)
LIST size=0: (empty)
```

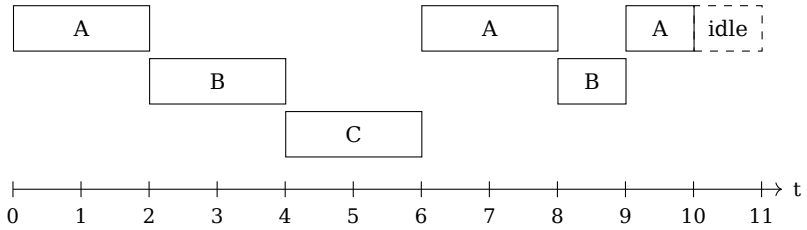
Хайлт, хэш хүснэгт, мод (examples/search.txt, холбогдох мөрүүд)

```
OK list find 17/3/1998 = index 1
OK list find 1/1/2001 = not found
OK queue find 20 = position 1
OK stack find 1 = position 2
OK hash put ab -> 7 (bucket=3, size=5, collision)
OK hash put ba -> 8 (bucket=3, size=6, collision)
HASH buckets=4 size=6 collisions=2: b0=[dave:11] b1=[carol:9] b2=[alice:1234] b3=[bob:5678
    -> ab:7 -> ba:8]
OK hash get ba = 8 (bucket=3, steps=3)
OK hash get bob = 5678 (bucket=3, steps=1)
OK hash get zed = not found (bucket=3)
OK hash put bob = 0 (updated, bucket=3)
ERR hash: key and value must be 1..12 letters, digits or _
TREE size=5 height=5 inorder: 1, 2, 3, 4, 5
ERR tree: mode can only be changed when the tree is empty
TREE(avl) size=7 height=3 inorder: 1, 2, 3, 4, 5, 6, 7
TREE(avl) size=6 height=3 inorder: 1, 2, 3, 5, 6, 7
```

Scheduler: гараар тооцоо ба програмын гаралт

Round Robin, квант 2; A=5, B=3, C=2 тик, бүгд t=0-д ирсэн. Тик t нь $[t, t+1)$ хугацааны завсар тул t=5 дээр дууссан процессын дуусах хугацаа 6 болно.

Хүснэгтийн зорилго: доорх хүснэгт нь гараар бодсон дундаж хүлээлтийг (5.00) C программын бодит `sched` гаралттай тулгаж, scheduler-ийн алгоритм зөв хэрэгжсэнийг онол ба практикийн хоёр тусдаа эх сурвалжаар нотлоход зориулагдсан.



Зураг 6 – Round Robin (q=2) Gantt диаграм: A, B, C процессийн CPU эзэмшсэн завсрууд [0,2) A, [2,4) B, [4,6) C, [6,8) A, [8,9) B, [9,10) A, [10,11) idle.

Процесс	Сүүлийн тик	Дуусах хугацаа	Нийт үргэлжилсэн	Ажил	Хүлээсэн
C (P3)	5	6	6	2	4
B (P2)	8	9	9	3	6
A (P1)	9	10	10	5	5
				Дундаж хүлээлт	**5.00**

Ижил ажлыг FCFS-ээр ажиллуулахад A 0, B 5, C 8 тик хүлээж, дундаж хүлээлт 4.33 болно. Round Robin богино процесст түргэн хариу өгдөг ч энэ жишээнд дундаж хүлээлт FCFS-ээс их гарсан нь квант ба шилжилтийн өртөг юм.

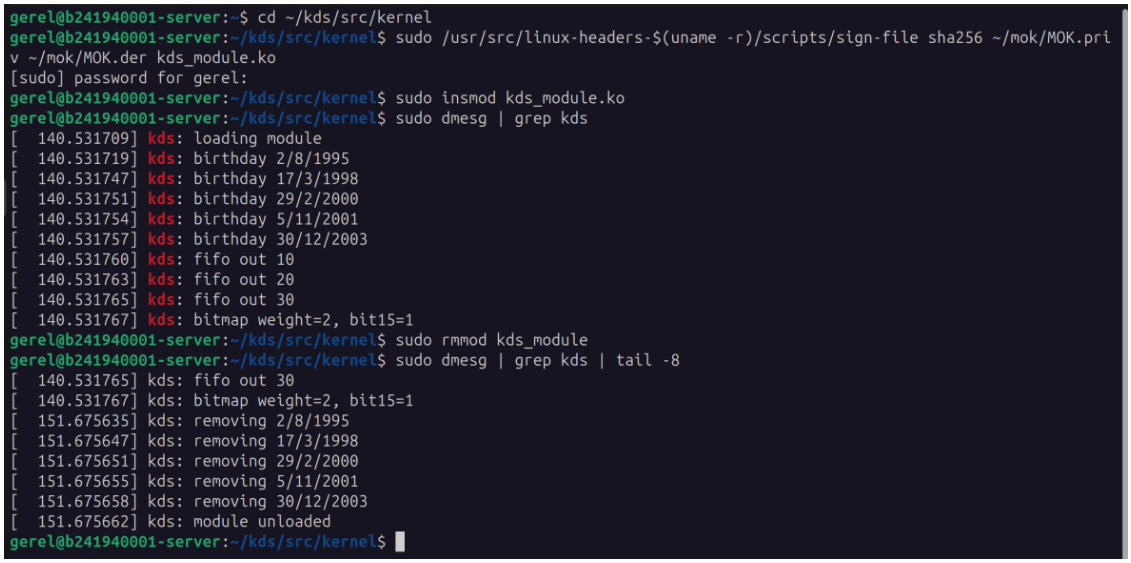
```
OK sched policy rr q=2
SCHED policy=rr(q=2) time=0 running=none ready=P1(A:5), P2(B:3), P3(C:2) done=0
TICK t=0 P1(A) remaining=4
TICK t=1 P1(A) remaining=3 preempted
TICK t=2 P2(B) remaining=2
TICK t=3 P2(B) remaining=1 preempted
TICK t=4 P3(C) remaining=1
TICK t=5 P3(C) remaining=0 finished
TICK t=6 P1(A) remaining=2
TICK t=7 P1(A) remaining=1 preempted
TICK t=8 P2(B) remaining=0 finished
TICK t=9 P1(A) remaining=0 finished
TICK t=10 idle
SCHED policy=rr(q=2) time=11 running=none ready=(empty) done=3
```

Буруу оролтын бодит гаралт (examples/errors.txt)

```
ERR list: day out of range for month          (31/4/2000)
ERR list: day out of range for month          (29/2/1900 - өндөр жил биш)
ERR list: month must be 1..12
ERR list: year must be 1900..2100
ERR list: date must be 3 numbers <day> <month> <year>, not d/m/y   (0/5/2000 ба 10/0/2000)
ERR list: day out of range for month          өдөр( 0)
ERR list: month must be 1..12                 сар( 0)
ERR list: usage <op> <day> <month> <year>
```

```
ERR list: 1/1/2000 not found
ERR list: missing operation
ERR unknown structure 'frob'
ERR queue: underflow (empty)
ERR stack: underflow (empty)
ERR bitmap: size must be 1..128               (0 ба 129)
ERR bitmap: index 99 out of range 0..31
ERR tree: duplicate 5
ERR tree: 9 not found
```

Kernel module



Зураг 1 – Kernel module-ийн өмнөх хувилбарын (birthday, kfifo, bitmap) бодит ачаалалт: Ubuntu 24.04, kernel 6.11.0-17-generic, MOK гарын үсэгтэй.

Part I/II нэмэгдсэн хувилбарын хүлээгдэх dmesg гаралт (`insmod kds_module.ko start=15`). Collatz гишүүдийг хэрэглэгчийн орчинд тооцоолж шалгасан: start=25 үед 24 гишүүн, start=15 үед 18 гишүүн.

```
kds: loading module (start=15)
kds: color red=255 blue=0 green=0
kds: color red=0 blue=255 green=0
kds: color red=0 blue=0 green=255
kds: color red=128 blue=64 green=32
kds: collatz 15
kds: collatz 46   23   70   35   106   53   160   80   40   20   10   5   16   8   4   2
kds: collatz 1
kds: collatz start=15 terms=18
kds: birthday 2/8/1995 ... 30/12/2003, fifo out 10 20 30, bitmap weight=2, bit15=1
(rmmod) kds: removing color ... / removing collatz 15 ... 1 / removing birthday ... /
    module unloaded
(insmod start=0) kds: start must be 1..1000000 (got 0)   -> insmod: Invalid parameters
```

9. Хэлэлцүүлэг

Бүх буруу оролт тодорхой мессеж өгч, програм нэг ч удаа унасангүй. ASan/UBSan санах ойн алдаа, leak мэдээлээгүй (Linux CI дээр; macOS 26 дээр `SAN=0` ажиллуулсан нь зөвхөн sanitizer шалгалтыг хасна). Веб апп нь бүтцийн логикийг дахин бичсэн хуулбар биш, зөвхөн C программаас ирсэн `--json` төлөвийг зурдаг тул «зураг бодит ажиллагаанаас зөрөх» асуудал угаасаа гарч ирэх боломжгүй; үүнийг 40 000 санамсаргүй командад текст гаралт ба `--json` гаралтын мессеж яг таарч байгаагаар баталгаажуулсан (мөн энэ нь веб апп ямар JSON-г хүлээж авч байгааг шалгасан гэсэн үг). Эрэмбэлсэн оролтод *BST*-ийн өндөр n-тэй тэнцүү болж, *AVL*-ийнх $\lg n$ орчим байсан нь онолтой таарч байна. Хэш хүснэгтэд *collision* нэмэгдэх тусам хайлтын алхам өссөн нь мөн онолтой нийцнэ.

Хязгаарлалт:

1. Kernel module-ийн өмнөх хувилбарыг нэг машин дээр (kernel 6.11.0-17) ачаалж туршсан. Өөр kernel хувилбарт дахин build хийх шаардлагатай. Secure Boot идэвхтэй үед гарын үсэггүй модуль ачаалагдахгүй тул MOK түлхүүрээр гарын үсэг зурсан.
2. Kernel module-ийн шинэчлэгдсэн hashtable, rbtree, `/proc/kds` хэсгийг жинхэнэ Linux дээр `insmod`-оор хараахан баталгаажуулаагүй (CI-ийн Linux job зөвхөн build хийдэг); `run_module_test.sh`-ийг бодит Linux машин дээр ажиллуулж шалгана. Red-black мод C цөмд (`rb` горим), singly жагсаалт (hlist) hash-ийн bucket-д хэрэгжсэн.
3. Хэш функц нь энгийн нийлбэр тул «ab», «ba» шиг үсгийн байр солигдсон түлхүүрүүд үргэлж collision үүсгэнэ. Bucket-ийн тоо auto resize-ээр 64 хүртэл ихэсдэг ч элементийн хязгаар 24-өөр бүдүүн дүрслэл болгож тоог хязгаарласан.
4. Scheduler-т бүх процесс үүсгэсэн мөчдөө ирнэ гэж үзсэн. I/O хүлээлт ба тэргүүлэх эрэмбэ байхгүй.
5. Веб апп зөвхөн дэлгэц дээр зурдаг, өөрөө логик агуулдаггүй тул интернэт/сервер шаардлагатай (хуучин нэг файлтай UI-гээс ялгаатай нь, тэр нь browser дотроо бүх логикыг давтдаг байсан). Веб аппыг локал ажиллуулахад `make`-аар `kds`-г урьдчилан build хийх шаардлагатай.
6. Queue, stack ба амьд процессын тоо 8, hash-ийн элемент 24-өөр хязгаарлагдсан. Энэ нь дүрслэлд багтаах зорилготой.

10. Дүгнэлт

Сурах бичгийн 3-р бүлгийн «Kernel Data Structures» төслийн хоёр хэсэг (color жагсаалт, `start` параметртэй Collatz жагсаалт) болон 1.9-ийн бүтэц бүрийг хэрэгжүүлсэн: холбоост жагсаалт, дараалал, стек, BST ба тэнцвэржсэн мод, хэш хүснэгт, bitmap. Нэмэх, устгах, хайх үйлдлийн дараах төлөвийг CLI (текст) болон веб хуудас (граф) хоёуланд харуулж чадсан бөгөөд хоёулаа яг нэг C программаас ирсэн мэдээлэлд тулгуурладаг. Нүдээр харуулсан: энгийн мод эрэмбэлсэн оролтод нэг шугам болдог (өндөр 7) бол *тэнцвэржссэн* мод өндрөө 3 хэвээр хадгалдаг, хэш хүснэгтэд *collision* үүсэхэд хайлтын алхам нэмэгддэг. Зөв ажиллагааг 8 скрипт, stress тест, AVL, red-black ба min-heap-ийн инвариант шалгалт, текст==JSON харьцуулалт (40 000 санамсаргүй команд), GitHub Actions дээрх автомат CI болон жинхэнэ Linux дээр ачаалсан kernel module-аар нотолсон. Нэмэлтээр «Processes» сэдвийн PCB ба ready queue ашиглан FCFS ба Round Robin-ийг хүлээлтийн хугацаагаар хэмжиж харьцуулсан.

Ашигласан материал

1. A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018 – Chapter 1 Introduction, 1.9 Kernel Data Structures (p. 36); Chapter 3 Processes.
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009 – Part III Data Structures (хайлт, BST, тэнцвэржсэн мод, hash table).
3. G. M. Adelson-Velsky, E. M. Landis. “An algorithm for the organization of information.” *Doklady Akademii Nauk SSSR*, vol. 146, 1962, pp. 263–266 (AVL тэнцвэржсэн модны анхны алгоритм).
4. D. P. Bovet, M. Cesati. *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA: O’Reilly Media, 2005 – процессын удирдлага, kernel-ийн холбоост жагсаалт (`list_head`) бүтэц.
5. F.CSM302 «Бие даалт 1» удирдамж.
6. Linux kernel эх код ба баримт: `include/linux/list.h`, `include/linux/kfifo.h`, `include/linux/bitmap.h`, `include/linux/moduleparam.h`, `include/linux/hashtable.h`, `include/linux/rbtree.h`; Documentation/core-api/kernel-api.rst.
7. The Linux Kernel Module Programming Guide (module_param, insmod/rmmod).
8. AI хэрэгсэл: Claude (Anthropic) – тайлангийн ноорог, C программын

`--json` горим, веб апп (Next.js), CI/release тохиргоо, тест скрипт, баримт бичгийн засварт ашигласан. Бүх кодыг баг ажиллуулж, тестээр шалгасан.

Хавсралт А. Ажлын төлөвлөгөө ба гишүүдийн оролцоо

Хүснэгтийн зорилго: доорх товч мэдээллийн хүснэгт нь төслийн нэр, код, хэл/орчныг нэг дороос олж харахад зориулагдсан – шалгуулагч эсвэл бусад хөгжүүлэгч төслийг эхлээд ямар орчинд ажиллуулах шаардлагатайг шууд мэдэх боломжтой.

Талбар	**Мэдээлэл**
Хуваарилалтын код	V-1-4
Төсөл	Kernel Data Structures
Repository	github.com/grl059/OS_BD1
Хэл / сан / орчин	C (gcc 13), TypeScript/React (Next.js), kernel C; Python 3, Node.js (тест, build); Ubuntu 24.04
Үндсэн дүрслэл	Kernel structure graph – веб апп (Next.js), 4 хуудастай (Нүүр, Чөлөөт горим, Хичээл, Тухай)

Хүснэгтээс харахад төсөл нь C, TypeScript/React, kernel C гэсэн гурван орчинд зэрэг ажилладаг, тул доорх хэсэгт гишүүн бүрийн хувь нэмрийг яг аль хэл/орчинд хийсэн эсэхээр нь ялган харуулав.

Хүснэгтийн зорилго: гишүүн тус бүрийн хариуцсан онол/туршилтын ажил, тэдгээрийн гаргасан бодит файл/commit-ийг хослуулж, баг даалгаврыг хэрхэн хуваасан, нэг нэгнээ давхардуулаагүйг нотолгоотойгоор харуулахын тулд оруулсан.

Гишүүн	Хариуцах ажил	Гаргасан файл / commit
Н. Ням-Од	Онолын судалгаа: сурах бичгийн Chapter 1, 1.9 Kernel Data Structures (p. 36) ба 3-р бүлгийн Programming Project, list_head ба container_of; тайлангийн 2–3-р хэсэг	2026-10-27: UI граф, дэлгэцийн зураг, Report/Manual PDF, V-1-4 – web/src/components/stage.tsx, docs/report_src/*, docs/V-1-4_*.pdf
А. Гэрэл	Repository удирдлага, веб апп (web/, Next.js) ба C программын `--json` горим, CI/release тохиргоо, C програм ба kernel module-ийг нэгтгэх, kernel module-ийг өөрийн Linux сервер дээр build хийж insmod/rmmod-оор турших; тайлангийн 4–6-р хэсэг	2026-10-20…27: first commit, UI, Report zasav, Example, Code buten bolgov, JSON mode/web app/CI
А. Номин	Туршилтын өгөгдөл ба хүлээгдэх гаралт: 5 огнооны жагсаалт (textbook), бүх бүтцийн давхар тест (all_structures), буруу/захын ороолт (errors.txt: 31/4, 29/2/1900, сар 13, жил 1800, bitmap 0 ба 129, индекс −1 ба 99, давхардсан tree утга). Мөр бүрийн ERR мессежийг гараар уншиж `tests/run_tests.sh`-ийн жагсаалтад нэмсэн; тайлангийн 7-р хэсэг	examples/textbook.txt, examples/all_structures.txt, examples/errors.txt, tests/expected_errors.out, tests/run_tests.sh
Э. Мягмаршагай	Процесс ба scheduler: A(5), B(3), C(2) процессийн RR (квант 2) ба FCFS хуваарийг хүснэгтээр гараар бодоод дуусах хугацаа 6/9/10, дундаж хүлээлт 5.00 ба 4.33-ыг `kds`-ийн гаралттай тулгасан; 8 процессийн `scheduler\_full`-д 9 дэх процесс татгалзагдахыг шалгасан; тайлангийн 8-р хэсэг	examples/scheduler.txt, examples/scheduler_full.txt, tests/expected_scheduler.out, tests/expected_scheduler_full.out
О. Хүслэн	Manual (хэрэглэгчийн ба operational заавар) бичиж, хэсэг бүрийг дагаж өөр компьютер дээр `make`, `make test`, `make check`, `npm install`, `npm run dev`-ийг гүйцэтгэж, зааварт бичсэн хүлээгдсэн гаралттай таарч байгааг гараар тулгаж шалгасан	docs/latex/manual.tex, docs/report_src/manual.html, docs/V-1-4_Manual.pdf
Ч. Түмэншагай	Hash map ба AVL модны туршилт: «ab»/«ba» түлхүүрийн collision, bucket доторх хайлтын алхам, эрэмбэлсэн 1–7 ороолтод BST өндөр 7 ба AVL өндөр 3-ыг гараар шалгасан; веб аппын хуудас бүрийн монгол тайлбар; тайлангийн 6, 9, 10-р хэсэг	examples/search.txt, tests/avl_check.py, web/src/lib/levels.ts, web/src/app/about/page.tsx

Тэмдэг: Ажил тус бүрийг багийнхан хамтдаа хийж, бие биенийхээ гаралтыг шалгасан ч GitHub дээрх commit/push-ийг нэг гишүүн төвлөрүүлж хийдэг байсан тул `git log`-д гишүүн бүрийн нэр тус тусдаа гаргаагүй. Үүнийг `git shortlog`-ийн оронд дээрх файл, тестийн нэрээр баталгаажуулсан (жишээ нь `make test`-ийн 9 PASS мөр, `tests/expected\_*.out`-ын гаралт).

Хүснэгтийн зорилго: эцсийн шалгалтаар ажиллуулсан 5 тестийг нэг хуудсанд товч нэгтгэж, баг даалгаврыг хамгийн сүүлд нэг бүрчлэн шалгаж байгаагаа баталгаажуулахад зориулсан.

Тест	Оролт / нөхцөл	Хүлээгдсэн үр дүн
1	examples/textbook.txt	5 огноо дарааллаараа, устгасны дараа хоосон
2	examples/search.txt	Хайлтын индекс, hash collision 2, BST өндөр 5, AVL өндөр 3
3	examples/scheduler.txt	RR дундаж хүлээлт 5.00, FCFS 4.33
4	examples/errors.txt	Мөр бүр ERR, програм унахгүй
5	insmod kds_module.ko start=15	4 color, 18 гишүүнтэй Collatz; start=0 → татгалзана

Эдгээр 5 тест нь тайлангийн 2-8-р хэсэгт дурдсан бүх шаардлагын (өгөгдлийн бүтэц, алдааны шалгалт, scheduler, kernel module) эцсийн, нэгтгэсэн баталгаажуулалт болж байгаа бөгөөд §8-д дурдсан бусад автомат тестүүд (stress, fuzz, avl_check, rb_check, CI)-тэй хамтдаа төслийн бүх хэсгийг давхар шалгасан гэсэн үг.